🧠 A Developer's Guide to Building Strategies in MQSMaster

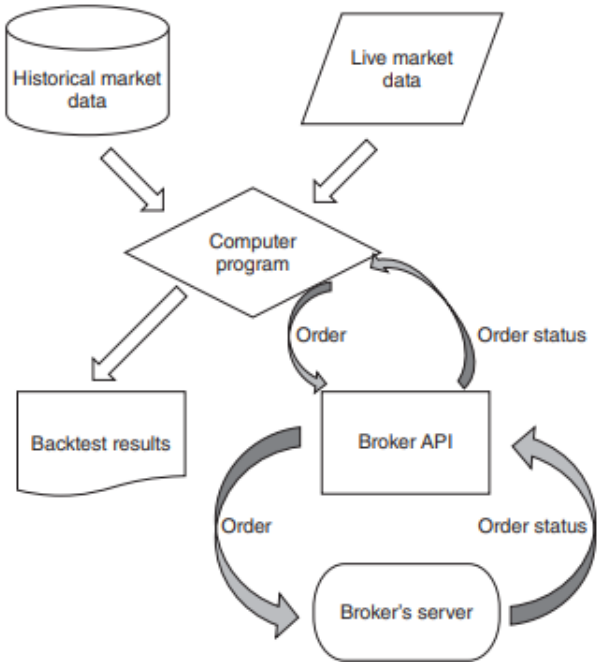
Status	Done
Created time	October 25, 2025 11:25 AM
Course	Python Cheat Sheet
TODO	- add visualise backetest - add kernal update
Type	Docs

This document provides a complete walkthrough for creating, configuring, and backtesting a quantitative trading strategy within the MQSMaster framework.

At a high level, a backtest in MQSMaster is an event-driven simulation. For a chosen period and interval:

1. Data is loaded for all configured tickers.
2. The engine advances time in discrete steps (INTERVAL seconds).
3. For each step and each strategy:
    - A StrategyContext is constructed with current market and portfolio state.
    - Indicators are updated with the latest data.
    - Your strategy's `OnData(context)` is called to make decisions.
    - The engine sizes and executes orders (with slippage) and updates cash/positions.
4. Metrics and portfolio state are updated and persisted during the run.
5. The simulation continues until `end_date`.

Multiple strategies can run simultaneously; the engine orchestrates their calls independently at each time step.



Quickstart

> **If you're in a hurry:**
>
> - Put your strategy in a `**src/portfolios/portfolio_(N)**/...' directory with `**config.json**` and `**strategy.py**`.
> - Import your strategy class in `**src/main_backtest.py**` and add it to `**portfolio_classes**` in the **backtest_engine.setup()**.
>
> ```
> from portfolios.portfolio_dummy.strategy import Crossove
> rRmiStrategy
> ```
>
> - Run:
>
> ```
>     python -m src.main_backtest
> ```
>
> **Quick Checklist**

- Created `src/portfolios/portfolio_N/` with `config.json` and `strategy.py`

- Registered indicators in `**__init__**` and checked `**.IsReady**` in `**OnData()**`
- Imported strategy in `**src/main_backtest.py**`
- Added strategy class to `portfolio_classes=[...]`

```
backtest_engine.setup(
            portfolio_classes=[CrossoverRmiStrategy],
            start_date="2024-01-01",
            end_date="2024-06-01",
            initial_capital=1000000.0,
            slippage=0.000001 # 0.1 basis point
        )
```

- Run and reviewed logs/results in '**src/visualize_backtests.ipynb**

```
python -m src.main_backtest
```

Getting Started: Setting Up Your Backtest

The main entry point for running a backtest is the src/main_backtest.py file.This is where you define the core parameters of your simulation. To run a backtest for your new strategy, you need to modify main_backtest.py file in two places:

1.1) Import Your Strategy Class: Add an import statement at the top of the file that has your strategy class available along with its path as in the following example.

Eg:

```
from portfolios.portfolio_dummy.strategy import CrossoverRmiS
trategy
```

1.2) Configure the Engine setup:

update this part of the code with your desired strategy, start date, end date, initial capital, slippage(currently in basis points)

```
backtest_engine.setup(
            portfolio_classes=[CrossoverRmiStrategy],
            start_date="2024-01-01",
            end_date="2024-06-01",
            initial_capital=1000000.0,
            slippage=0.000001 # 0.1 basis point
        )
```

You can run multiple strategies at the same time, all you need to do is to import all your strategies to **main_backtest.py** as mentioned in step “1.1” and all you have to do is add the strategies you want to test in the “**portfolio_classes**” list.

```
portfolio_classes=[CrossoverRmiStrategy, MomentumStrat
egy]
```

▼ Initalizing Single Strategy

```
# This is where backtests are setup.
# Simply add you portfolio class to the list of portfolio
classes in the `setup` method, line 28.
```

```
# High level overview of how to set up a backtest:

# 1. Load your portfolio through imports such as `from por
tfolios.portfolio_1.strategy import VolMomentum`
# 2. Set up the backtest engine with the database connecto
r and backtest executor.
# 3. Call the `setup` method with the portfolio class, sta
rt date, end date, and initial capital.
# 4. Call the `run` method to execute the backtest.
# src/main_backtest.py

import logging
from common.database.MQSDBConnector import MQSDBConnector
from portfolios.portfolio_dummy.strategy import CrossoverR
miStrategy
from backtest.backtest_engine import BacktestEngine

logging.basicConfig(level=logging.ERROR, format='%(asctim
e)s - %(name)s - %(levelname)s - %(message)s')

def main():
    """
    Main entry point for the MQS Trading System backtests.
    """
    try:
        dbconn = MQSDBConnector()

        backtest_engine = BacktestEngine(db_connector=dbco
nn, backtest_executor=None)

        backtest_engine.setup(
            portfolio_classes=[CrossoverRmiStrategy],
            start_date="2024-01-01",
            end_date="2024-06-01",
            initial_capital=1000000.0,
            slippage=0.000001 # 0.1 basis point
```

```
        )

        backtest_engine.run()

    finally:
        logging.info("===== DONE =====")

if __name__ == '__main__':
    main()
```

▼ Initalizing Multiple Strategies

```
# This is where backtests are setup.
# Simply add you portfolio class to the list of portfolio
classes in the `setup` method, line 28.

# High level overview of how to set up a backtest:

# 1. Load your portfolio through imports such as `from por
tfolios.portfolio_1.strategy import VolMomentum`
# 2. Set up the backtest engine with the database connecto
r and backtest executor.
# 3. Call the `setup` method with the portfolio class, sta
rt date, end date, and initial capital.
# 4. Call the `run` method to execute the backtest.
# src/main_backtest.py

import logging
from common.database.MQSDBConnector import MQSDBConnector
from portfolios.portfolio_dummy.strategy import CrossoverR
miStrategy
from portfolios.portfolio_dummy.strategy import RSI <-- ad
d stratrgy
from backtest.backtest_engine import BacktestEngine

logging.basicConfig(level=logging.ERROR, format='%(asctim
```

```
e)s - %(name)s - %(levelname)s - %(message)s')

def main():
    """
    Main entry point for the MQS Trading System backtests.
    """
    try:
        dbconn = MQSDBConnector()

        backtest_engine = BacktestEngine(db_connector=dbco
nn, backtest_executor=None)

        backtest_engine.setup(
            portfolio_classes=[CrossoverRmiStrategy, RSI],
<---#Add to list
            start_date="2024-01-01",
            end_date="2024-06-01",
            initial_capital=1000000.0,
            slippage=0.000001 # 0.1 basis point
        )

        backtest_engine.run()

    finally:
        logging.info("===== DONE =====")

if __name__ == '__main__':
    main()
```

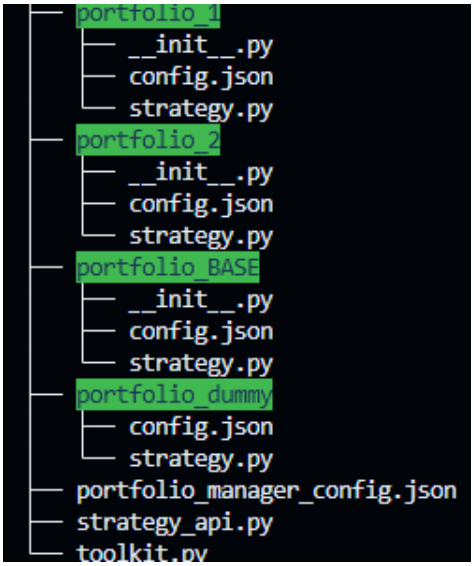
Once this is configured, you can run the backtest from your terminal:

```
python -m src.main_backtest
```

Creating a New Strategy: File Structure

Each strategy folder must contain two essential files:

1. **config.json**: This file holds all the static configuration for your strategy.
2. **strategy.py**: This file contains the Python class with your trading logic.



Add a .txt of your strategy in the file for easy identification
i.e. Momentum.txt, VolMomentum.txt

The Config.json File

How it’s used during backtest:

- The engine loads the universe (`TICKERS`) and requests `DATA_FEEDS`.
- The clock advances in steps of `INTERVAL`.
- Up to `LOOKBACK_DAYS` of data is available to indicators and `asset.History(...)`.
- `WEIGHTS` informs the executor how to size orders when you call `context.buy`/`context.sell`.

This JSON file defines the universe and key parameters for your strategy.

- **PORTFOLIO_ID (string):** A unique ID for your portfolio, matching the folder number.
- **TICKERS (list of strings):** The list of asset tickers your strategy will trade (e.g., ["AAPL", "TSLA"]).
- **WEIGHTS (dictionary):** Defines the target weight for each ticker. This is used by the trading executor to calculate trade sizes.
- **INTERVAL (integer):** The time in **seconds** between each call to your strategy's OnData method (e.g., 3600 for one hour).
- **LOOKBACK_DAYS (integer):** The number of days of historical data that will be available in the StrategyContext at each step of the backtest.
- **EXCH(EXCHANGE: STRING):** the exchange of the tickers in the portfolio DEFAULT “NASDAQ”

```
{
    "PORTFOLIO_ID": "4",
    "TICKERS": ["AAPL", "TSLA", "NVDA", "MSFT"],
    "WEIGHTS": {
        "AAPL": 0.25,
        "TSLA": 0.25,
        "NVDA": 0.25,
```

```
        "MSFT": 0.25
    },
    "INTERVAL": 3600,
    "LOOKBACK_DAYS": 50
}
```

The strategy.py File: Core Concepts

This file is where your strategy's logic lives.

The Strategy Class

Your strategy must be a Python class that inherits from BasePortfolio.

The __init__ Method

The constructor is where you initialize your strategy. It's primarily used for setting up stateful indicators.

- You **must** call super().__init__(...) to ensure the base class is properly configured.
- This is the ideal place to call self.RegisterIndicatorSet() to create your indicators.

The OnData(self, context) Method

This is the **main entry point for your strategy logic**. The backtesting engine will call this method periodically based on the INTERVAL you set in your config file. It receives a single, powerful argument: the context object.

▼ Strategy.py Example

```
from src.portfolios.indicators.base import Indicator
from src.portfolios.portfolio_BASE.strategy import BasePor
tfolio
from src.portfolios.strategy_api import StrategyContext
```

```python
class CrossoverRmiStrategy(BasePortfolio):
    """
    A showcase strategy demonstrating advanced use of the StrategyContext API,
    including portfolio-level risk management, data transformation with the
    .toolkit accessor, and dynamic position logic.
    """
    def __init__(self, db_connector, executor, debug=False, config_dict=None, backtest_start_date=None):
        super().__init__(db_connector, executor, debug, config_dict, backtest_start_date)
        self.logger = logging.getLogger(f"{self.__class__.__name__}_{self.portfolio_id}")

        # --- 1. Define All Indicators ---
        indicator_definitions = {
            "fast_sma": ("SimpleMovingAverage", {"period": 20}),
            "slow_sma": ("SimpleMovingAverage", {"period": 50}),
            "rmi": ("RelativeMomentumIndex", {"period": 14, "momentum_period": 4})
        }
        self.RegisterIndicatorSet(indicator_definitions)

        # --- State Tracking ---
        self._previous_sma_values: Dict[str, Dict[str, float]] = {
            ticker: {"fast": 0.0, "slow": 0.0} for ticker in self.tickers
        }

    def OnData(self, context: StrategyContext):
     portfolio = context.Portfolio
```

```
        # Risk Rule: Don't open new long positions if cash
is less than 10% of total value
        is_risk_off = portfolio.cash < (portfolio.total_va
lue * 0.10)
        if is_risk_off:
            self.logger.warning("Risk-Off Mode: Cash is lo
w. No new long positions will be opened.")

        # --- 2. Loop through assets for trading logic ---
        for ticker in self.tickers:
            asset = context.Market[ticker]

        # Get indicator instances
          fast_sma = self.fast_sma[ticker]
          slow_sma = self.slow_sma[ticker]
          rmi = self.rmi[ticker]

          # Wait until all indicators are ready
            if not all([asset.Exists, fast_sma.IsReady, sl
ow_sma.IsReady, rmi.IsReady]):
                continue


              ... strategy logic
```

Always guard on indicator readiness to avoid using uninitialized values:

```
if not all([asset.Exists, fast_sma.IsReady, slow_sma.I
sReady, rmi.IsReady]):
    continue
```

Indicators

The framework includes a powerful, stateful indicator system that is both efficient and easy to use. The recommended way to create indicators is in your strategy's __init__ method.

- **self.RegisterIndicatorSet(indicator_definitions: Dict)**:
  - **Description**: The primary helper to initialize a set of indicators for every ticker in your portfolio.
  - **Argument**: A dictionary where keys are the desired attribute names (e.g., "fast_sma") and values are a tuple containing the indicator's class name (as a string) and a dictionary of its parameters.
- you can access your indicators in OnData like this:
  - **self.fast_sma['AAPL'].**

The Indicator Object

Each indicator instance has two key properties:

- **.IsReady**:
  - **Type**: bool
  - **Description**: True once the indicator has processed enough data to produce a valid value. **Always check this before using an indicator.**
- **.Current**:
  - **Type**: float

Description

: Returns The latest value of the indicator. (i.e. RMI = 46)

Example: Initializing Indicators

```
        # --- 1. Define All Indicators ---
        indicator_definitions = {
            "fast_sma": ("SimpleMovingAverage", {"period": 2
0}),
            "slow_sma": ("SimpleMovingAverage", {"period": 5
0}),
```

```
            "rmi": ("RelativeMomentumIndex", {"period": 14,
"momentum_period": 4})
        }
        self.RegisterIndicatorSet(indicator_definitions)

        # --- State Tracking ---
        self._previous_sma_values: Dict[str, Dict[str, floa
t]] = {
            ticker: {"fast": 0.0, "slow": 0.0} for ticker in
self.tickers
        }


        def OnData(self, context: StrategyContext):
              portfolio = context.Portfolio

          for ticker in self.tickers:
            asset = context.Market[ticker]

            # Get indicator instances
            fast_sma = self.fast_sma[ticker]
            slow_sma = self.slow_sma[ticker]
            rmi = self.rmi[ticker]
```

Best practices:

- Always check `**.IsReady**` before using `**.Current**`.
- Keep indicator periods and data cadence consistent with `INTERVAL`.
- Use `LOOKBACK_DAYS` large enough to cover your longest warmup window.

The strategy_api: Your Primary Tool 🛠️

> The **context** object is your gateway to all market data, portfolio information, and trading functions.

context.time:

- **Type**: datetime.datetime
- **Description**: The current timestamp of the simulation step.

context.Market:

The Market object provides access to all market data for your configured tickers. You access data for a specific asset using dictionary-style syntax.

- **Usage**: asset = context.Market['AAPL']
- **Returns**: An Asset DataFeed object.

The AssetData Object:

This object represents a snapshot of a **single asset** at **context.time.**

- **asset.Exists**:
  - **Type**: bool
  - **Description**: True if there is valid market data for this asset at the current time.
- **asset.Close, asset.Open, asset.High, asset.Low, asset.Volume**:
  - **Type**: float
  - **Description**: The latest OHLCV data points for the asset.
- **asset.History(lookback_period: str)**:
  - **Description**: Fetches a historical slice of data for this asset.
  - **Argument**: A pandas-compatible time string (e.g., "30d" for 30 days, "90m" for 90 minutes).
  - **Returns**: A pandas.DataFrame indexed by timestamp with OHLCV columns.

context.Portfolio

The Portfolio object provides a clean interface to your current trading account's state.

- **Usage**: portfolio = context.Portfolio
- **Returns**: A PortfolioManager object.

The PortfolioManager Object

- **portfolio.cash**:
  - **Type**: float
  - **Description**: The current amount of cash available.
- **portfolio.total_value**:
  - **Type**: float
  - **Description**: The total equity of the portfolio (cash + value of all positions).
- **portfolio.positions**:
  - **Type**: Dict[str, float]
  - **Description**: A dictionary mapping tickers to their current quantity held. An empty dictionary means you are flat.
- **portfolio.get_asset_weight(ticker: str, current_price: float)**:
  - **Description**: Calculates the current weight of an asset as a percentage of the total portfolio value.
  - **Returns**: float (e.g., 0.1 for 10%).

Trading Functions

- **context.buy(ticker: str, confidence: float = 1.0)**
  - Executes a buy order. The system calculates the trade size based on your config WEIGHTS.
- **context.sell(ticker: str, confidence: float = 1.0)**
  - Executes a sell order (to close a long position or open a short).

Order Sizing, Execution, and Slippage

Order sizing is derived from your `WEIGHTS` and current portfolio composition. `confidence` scales the action toward (or away from) target weight.

When you call `context.buy("SPY")` or `context.sell("QQQ")`, the engine:

1. Computes the current weight of the asset vs. the target from `WEIGHTS`.
2. Calculates the quantity needed to move toward the target weight, factoring in:
    a. Available cash (for buys) or position size (for sells).
    b. The optional `confidence` parameter (0.0 to 1.0).
3. Applies slippage to the fill price.
4. Updates `cash`, `positions`, and `total_value`.

Slippage

Slippage is a fractional price adjustment applied to fills:

- Buy fill ≈ price × (1 + slippage)
- Sell fill ≈ price × (1 - slippage)

Basis points (bp) mapping:

- 1 bp = 0.0001 (i.e., 0.01%)

Examples:

- 0.0001 = 1.0 bp
- 0.00001 = 0.1 bp
- 0.000001 = 0.01 bp

Choose a slippage representative of your market and bar interval.

Data Transformation with the .toolkit Accessor 🔬

The framework provides a custom .toolkit accessor on all pandas DataFrames and Series, allowing you to easily apply common financial data transformations.

- **Usage**: Simply call .toolkit on a Series or DataFrame, such as one returned by asset.History().

Available Functions:

- **.toolkit.winsorize(limits: List[float])**:
  - **Description**: Caps outliers in the data to the specified quantiles.
  - **Returns**: A pandas.Series or pandas.DataFrame.
- **.toolkit.gaussian_smooth(sigma: float)**:
  - **Description**: Applies a Gaussian smoothing filter to a Series.
  - **Returns**: A pandas.Series.

Example: Using the Toolkit

```
# In your OnData method
history = context.Market['SPY'].History("30d")
returns = history['close_price'].pct_change().dropna()

# Remove outliers from the returns before calculating volatil
ity**
winsorized_returns = returns.toolkit.winsorize(limits=[0.05,
0.05])
volatility = winsorized_returns.std()
```

Putting It All Together: A Showcase Strategy

This is how the flow goes:(FYI: these are all class names)

Strategy ⇒ builds: StrategyContext ⇒ Fetches: MarketData ⇒ Instantiates: AssetData

||

initialized by the; **PortfolioManager**

Within each time step;

1. The engine advances the clock to the next bar time.
2. Market data for that bar is loaded and bound to `context.Market`.
3. Indicators are updated with new bar values (per ticker).
4. The engine builds a `StrategyContext` for each strategy.
5. Your strategy's `OnData(context)` is invoked:

- Read state, evaluate signals, call `buy`/`sell`.

6. Orders are sized and executed with slippage applied.
7. Portfolio state is updated (cash, positions, notional).
8. Metrics/logs are recorded.
9. Repeat until `end_date`.

Warmup and Lookbacks:

- Indicators will not be "ready" until they have seen enough bars.
- Use `LOOKBACK_DAYS`to ensure sufficient data for your signal horizon.

Running Multiple Strategies in Parallel

To compare or ensemble strategies:

- Import each strategy class in `src/main_backtest.py`.
- Pass all of them in `portfolio_classes=[...]`.
- Each strategy uses its own `PORTFOLIO_ID` and manages its own cash/positions.

- The engine steps time once, then invokes `OnData` for each strategy independently.

This enables A/B tests over the same market regime window using consistent assumptions.

Practical Tips and Gotchas

- Interval alignment: Ensure your `INTERVAL` matches the bar frequency of your market data.
- Indicator warmup: Use conservative `LOOKBACK_DAYS` to avoid cold-starts.
- Slippage sanity: Map bps to decimals correctly:
  - 1 bp = 0.0001, 0.1 bp = 0.00001, 10 bps = 0.001
- Weight-based sizing: If you set `WEIGHTS={"SPY": 1.0}`, a “buy” action targets 100% SPY allocation (budget and confidence permitting).
- Confidence scaling: Use `confidence` < 1.0 to move partially toward the target weight.

Defensive checks:

- Check `asset.Exists` before using OHLCV fields.
- Check indicator `.IsReady` before using `.Current`.
- try/except to handle missing data or symbol halts.

Troubleshooting:

- No orders? Verify `OnData` is being called (add logs), indicators are ready, and your signals trigger.
- Unexpected fill prices? Revisit `slippage` and confirm buy/sell direction logic.
- Cash negative? Your config should constrain sizes; review weight targets and confidence usage.

Glossary

- Bar/Interval: The time bucket for OHLCV aggregation. Controlled by `INTERVAL`.
- Basis Point (bp): 0.01%. In decimals, 1 bp = 0.0001.
- Weight: Fraction of total portfolio value allocated to a ticker (e.g., 0.5 = 50%).
- StrategyContext: A snapshot of market/portfolio state at a time step, plus trading helpers.

If data is missing for a ticker in the database go to src/orchestrator/backfill/Readme.md

To use the learn how to use the backfill cli to populate the dates

(only on fallRefactor branch use: **git switch fallRefactor**)